Four Bit Few Photon Fluorescence Lifetime Imaging: Compressing FPFLI to 4 Bit Weights with Near Full Precision Accuracy

Md Forhadul Islam
Independent study, London, United Kingdom
Code: https://github.com/forhadsde/fpfli-32bit-to-4bit-weights-near-full-precision

***Abstract*: Few photon fluorescence lifetime imaging (FPFLI), introduced by Xiao, Sapermsap, Chen and Li, uses two neural networks to recover sharp lifetime images from about ten photons per pixel. This independent study asks whether the whole pipeline can be made small enough for low power hardware such as an FPGA without losing accuracy. I first reproduce the published results, obtaining a mean absolute error (MAE) of 0.134 ns on real HEK293 cells. Plain rounding of the local lifetime estimator to 4 bit weights breaks it (1.045 ns). A layer sensitivity study traces the damage to the output layers. Quantisation aware training, distillation from the 32 bit teacher, learned step sizes and a few small layers kept at 8 bit recover a 4 bit estimator of 13.5 KB with 0.142 ns. The 58.2 MB image network is compressed to about 7.3 MB by distillation on images from a different SPAD camera, without training on the test images. The full pipeline with both networks at 4 bit reaches 0.139 ns, within 0.005 ns of full precision, while the weights shrink about eight times. The results are simulated in PyTorch with weight only quantisation, and the limitations are stated clearly.**

***Index Terms*: fluorescence lifetime imaging, quantisation, quantisation aware training, knowledge distillation, edge AI, FPGA**

I. Introduction

Fluorescence lifetime imaging (FLIM) measures how long molecules stay excited, which reveals their local environment. Fast FLIM is hard because each pixel receives very few photons. FPFLI addresses this with two networks [1]. A local lifetime estimator (LLE) reads the photon arrival histogram of each block of pixels and predicts one lifetime per block. A neural implicit image interpolation network (NIII) then turns the blocky lifetime map into a sharp full resolution map, guided by the intensity image.

Dr Li's group has also shown compact lifetime networks on FPGA hardware with reduced bit widths [2]. This motivated a simple question: how far can the FPFLI weights be compressed before accuracy suffers? Low bit weights reduce memory and could make on chip inference easier. My work in industry includes a weight quantisation pipeline for a large diffusion model, so I wanted to test the same ideas on a scientific imaging pipeline.

This report makes four contributions. First, it reproduces FPFLI with the released code and models. Second, it measures where 4 bit weights break the LLE. Third, it recovers a 4 bit LLE through retraining. Fourth, it compresses the NIII and evaluates the whole 4 bit pipeline. It is an independent study and not a claim of a new imaging method.

II. Setup

A. Networks and data

The LLE is a one dimensional ConvMixer (tauNet) with 27,272 convolution weights (106.5 KB at 32 bit). The NIII has two residual dense network encoders and an MLP decoder, with 14,553,600 weights (58.2 MB). I use the released FPFLI code, trained models and sample data: six synthetic images and one real HEK293 cell image with a reference, at about 10.2 photons per pixel. For NIII distillation only, I use the public Glasgow megapixel SPAD FLIM data [3], so the test images are never used for training the image network.

B. Quantisation methods

Weights are quantised per output channel and symmetrically. At 4 bit each channel uses 15 levels, from minus 7 to plus 7, with one scale. Activations stay at 32 bit. Quantisation aware training (QAT) rounds weights in the forward pass and passes gradients through the rounding with the straight through estimator [4]. Distillation trains the 4 bit student to copy the 32 bit teacher [5]. Learned step size quantisation trains the scale of each channel [6]. Mixed precision keeps a few small or sensitive layers at 8 bit, and BatchNorm statistics are recalibrated after training.

C. Evaluation

The metric is lifetime MAE in nanoseconds on the synthetic set (mean of six images) and on HEK293. All final numbers are recomputed from the saved models in one deterministic run on an NVIDIA GeForce RTX 3090.

III. Results

A. Reproduction and the 4 bit break

The reproduction matches the published behaviour: 0.134 ns on HEK293 and 0.161 ns on the synthetic set. Plain 8 bit rounding of the LLE costs a little (0.161 ns on HEK293). Plain 4 bit rounding breaks it: the error rises to 1.045 ns and the whole lifetime map reads too long.

TABLE I
LOCAL LIFETIME ESTIMATOR RESULTS (MAE, NS)

Model	Synthetic	HEK293
32 bit (published)	0.161	0.134
8 bit, plain rounding	0.165	0.161
4 bit, plain rounding	1.472	1.045
4 bit, mixed precision (14 layers 8 bit)	0.155	0.153
4 bit, QAT (V1)	0.259	0.185
4 bit, plus distillation (V2)	0.165	0.159
4 bit, plus learned scales (V3)	0.167	0.150
4 bit, plus four small 8 bit layers (V4)	0.167	0.142

TABLE II
IMAGE NETWORK AND WHOLE PIPELINE RESULTS (MAE, NS)

Model	Synthetic	HEK293
NIII 8 bit, plain rounding	0.161	0.134
NIII 4 bit, plain rounding	0.294	0.157
NIII 4 bit, distillation	0.162	0.129
NIII 4 bit, distillation, 3 layers 8 bit	0.158	0.129
NIII 4 bit, plus learned scales	0.157	0.132
Whole pipeline, both networks 4 bit	0.162	0.139

B. Which layers break

Rounding one layer at a time to 4 bit shows that the damage sits at the output end of the LLE. The pointwise convolution named ponitconv1 in the released code alone raises the HEK293 error to 0.607 ns, and ponitconv3 to 0.390 ns. Keeping the sensitive layers at 8 bit needed 14 of 21 layers (25.1 KB), which is barely smaller than plain 8 bit. Simple mixed precision was therefore not enough, and retraining was needed. A photon budget sweep also shows that FPFLI beats the centre of mass baseline at normal light levels, while near one photon per pixel the two become close.

C. Recovering a 4 bit LLE

Table I shows the retraining steps. QAT alone gives 0.185 ns on HEK293. Adding distillation from the 32 bit teacher with matched training data gives 0.159 ns. Learned scales give 0.150 ns. Keeping four tiny layers (392 weights) at 8 bit gives 0.142 ns from 13.5 KB.

D. Compressing the image network and the whole pipeline

Plain 8 bit rounding of the NIII is lossless on HEK293 (0.134 ns). Plain 4 bit rounding hurts moderately (0.157 ns on HEK293 and 0.294 ns on the synthetic set). Distillation on practice images from a different SPAD camera fully recovers it, giving 0.129 to 0.132 ns on HEK293 depending on the variant (Table II). Combining the V4 LLE with the best 4 bit NIII, the whole pipeline reaches 0.162 ns on the synthetic set and 0.139 ns on HEK293, with weights of 13.5 KB and about 7.3 MB (Fig. 1).

IV. DISCUSSION AND LIMITATIONS

The study shows that both FPFLI networks tolerate 4 bit weights once they are retrained with the right signals, and that the failure of plain rounding is concentrated in a few output layers. Distillation on data from another camera was enough to recover the image network, which is encouraging for settings where labelled data is scarce.

The limitations are important. Only weights are quantised, so activations are still 32 bit and this is not yet an integer pipeline. The results come from fake quantisation in PyTorch, so they show the accuracy effect but not speed or power, and nothing has run on an FPGA yet. The test set is small: one real image with a reference and six synthetic images, with one training seed per version, so small differences such as V3 against V4 may be noise. Versions were compared on the test images, and the HEK293 instrument response stored with the test file was used to build training decays for the LLE. Counting biases, BatchNorm and per channel scales, the 4 bit LLE is about 26 KB rather than 13.5 KB.

V. CONCLUSION AND NEXT STEPS

Retraining with quantisation aware training and distillation compresses the FPFLI pipeline about eight times while keeping the HEK293 error within 0.005 ns of full precision. The next steps are 8 bit activations so the pipeline becomes fully integer, running the 13.5 KB LLE on an FPGA to measure latency, power and accuracy on the board, and applying the same method to blood flow estimation from diffuse correlation spectroscopy.

ACKNOWLEDGEMENT

The FPFLI code, pretrained models and sample data belong to Xiao, Sapermsap, Chen and Li. This study would not have been possible without their open release.

REFERENCES

[1] D. Xiao, N. Sapermsap, Y. Chen and D. D. U. Li, "Deep learning enhanced fast fluorescence lifetime imaging with a few photons," *Optica*, vol. 10, no. 7, pp. 944 to 951, 2023.

[2] Z. Zang, D. Xiao, Q. Wang, Z. Jiao, Y. Chen and D. D. U. Li, "Compact and robust deep learning architecture for fluorescence lifetime imaging and FPGA implementation," 2023.

[3] V. Zickus *et al.*, "Fluorescence lifetime imaging with a megapixel SPAD camera and neural network lifetime estimation," *Scientific Reports*, vol. 10, 2020.

[4] Y. Bengio, N. Léonard and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

[5] G. Hinton, O. Vinyals and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[6] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy and D. S. Modha, "Learned step size quantization," in *Proc. ICLR*, 2020.

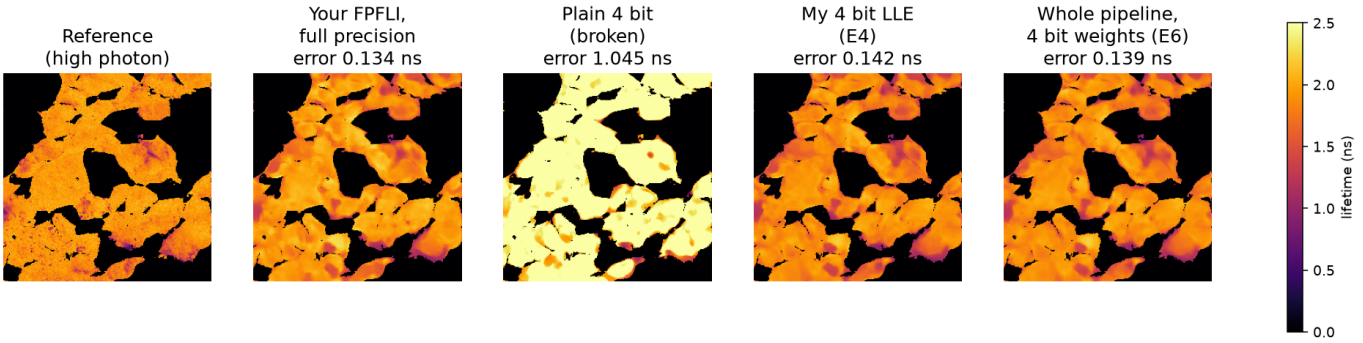


Fig. 1. HEK293 lifetime maps. From left: reference, published 32 bit FPFLI, plain 4 bit LLE, retrained 4 bit LLE, and the whole pipeline with both networks at 4 bit.

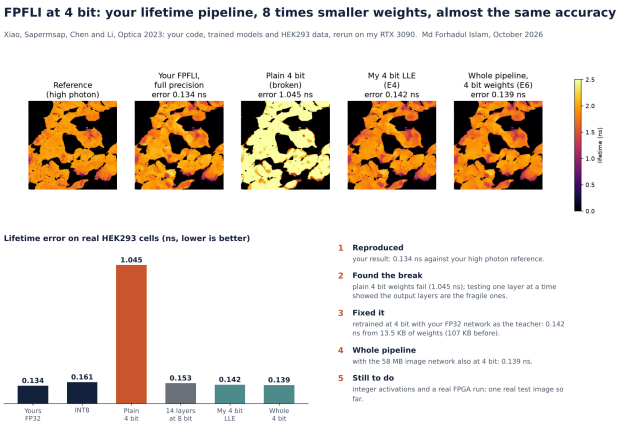


Fig. 2. Summary of the 4 bit results: error and weight memory for each stage.